DBnomics for Macroeconomic Data

A reproducible guide for teaching, MATLAB, Dynare, and Python

Gauthier Vermandel

21 August 2026

Repository. `https://github.com/Vermandel/dbnomics-teaching`
Purpose. This is a generic companion guide for courses using macroeconomic data. It explains how to find, download, document, and prepare series from DBnomics. It is not specific to one assignment or one model.

1 The reproducible workflow

DBnomics makes datasets from many providers available through a common web interface and API. It does not remove the need to understand the original source, definitions, units, seasonal adjustment, frequency, licence, or revision policy. The reliable sequence is:

1. Find a series in the DBnomics web interface and inspect its metadata.
2. Record its *provider*, *dataset*, and *series code*.
3. Call the API from a script, rather than committing a downloaded file.
4. Preserve the raw period label and missing values; transform them explicitly and document the choice.
5. Save a small, model-ready CSV only locally or as a build product.

An API endpoint has the form

`https://api.db.nomics.world/v22/series/PROVIDER/DATASET/SERIES?observations=1`.

For each empirical result, record the endpoint, access date, series frequency, provider, transformations, and treatment of missing values. Cite the original provider and DBnomics as appropriate.

2 Tested identifiers

The following endpoints were executed successfully on 21 August 2026. A live validation script is included in the repository. Identifiers can be revised by providers; rerun it before using examples in a new project.

Frequency	Provider / dataset	Series code
Monthly	ECB / EXR	`M.USD.EUR.SP00.A`: USD per euro spot average
Monthly	ECB / EXR	`M.GBP.EUR.SP00.A`: pound sterling per euro spot average
Quarterly	IMF / IFS	`Q.FR.NGDP_R_SA_XDC`: French real GDP, seasonally adjusted
Annual	WB / WDI	`A-NY.GDP.MKTP.KD-FRA`: French GDP, constant 2015 US dollars

The provider code for the World Bank example is `WB`. The intuitive spelling `WorldBank` was checked and returns HTTP 404, so it must not be used in code or teaching material.

3 Python: download first, then transform

The repository's Python examples require only the standard library. From a clone, first validate every advertised identifier:

```
python3 examples/python/validate_examples.py
```

To download one monthly series:

```
python3 examples/python/download_series.py \
  ECB/EXR/M.USD.EUR.SP00.A --output /tmp/eurusd.csv
```

Several series can be written into the same long, tidy file:

```
python3 examples/python/download_series.py \
  ECB/EXR/M.USD.EUR.SP00.A ECB/EXR/M.GBP.EUR.SP00.A \
  --output /tmp/exchange-rates.csv
```

The output has the columns `series`, `period`, and `value`. Keeping a long format makes the identity of each series explicit. The optional `-start` and `-end` arguments retain a lexical range of standard annual, monthly, or quarterly period labels.

`annual_growth.py` illustrates a transparent transformation: it downloads annual French GDP and calculates a percent change only when both adjacent observations are present. It leaves the first growth rate and values next to missing observations blank. Do not silently interpolate, forward-fill, or change frequency without a stated economic and statistical reason.

4 MATLAB and a Dynare pre-processing step

`examples/matlab/download_series.m` uses `webread` and `jsondecode` to create a table and, optionally, a CSV:

```
T = download_series('IMF','IFS','Q.FR.NGDP_R_SA_XDC', ...
    'examples/output/france-real-gdp-quarterly.csv');
T.date = parse_dbnomics_period(T.period);
```

`parse_dbnomics_period.m` explicitly recognises only the annual `YYYY`, monthly `YYYY-MM`, and quarterly `YYYY-Qq` formats used in the examples. It fails for an unfamiliar convention, which is safer than guessing dates. MATLAB represents unavailable numeric observations as `NaN`; decide and document any subsequent treatment.

`download_examples.m` produces local monthly, quarterly, and annual CSV files and writes a two-column quarterly example suitable as a transparent input stage before MATLAB/Dynare. A Dynare model should not assume an unlabelled CSV: document the variable order, units, sample, transformations, and any alignment or missing-data rule in the pre-processing script.

5 Finding and checking a series

1. Search `https://db.nomics.world/` by concept, country, and provider.
2. Read the title and dimensions; compare the frequency, unit, adjustment, and coverage with the intended model variable.
3. Copy the three identifiers from the page or API URL exactly.
4. Run a small request and inspect the first/last period and observation count. The repository validator does this for its examples.
5. Keep a human-readable data note alongside code: provider/dataset/series, URL, retrieval date, definitions, and all transformations.

Do not publish large downloaded datasets when a deterministic downloader is enough. If a project uses a restricted, changing, or manually curated input, document the access constraint and publish only what redistribution permits.

6 Checks before a release

- Run the live Python validator and the downloader from a clean clone.
- Confirm each API response returns the requested provider/dataset/series.
- Check date parsing at the intended frequency and preserve missing values.
- Rebuild this PDF from its LaTeX source.
- Keep only source, documentation, useful code, and the canonical PDF: no logs, caches, downloaded data, credentials, or local paths.